Test Specification

NestUp

What is tested, why, and what is not

Internet Technologies — Become a Full-Stack Engineer

RUNI CS 2026 · Final project

Live product **nestup-kappa.vercel.app**

Repository **github.com/nestupweb/nestup**

Suite: 97 test files · 740 tests · Vitest + React Testing Library (jsdom) + Playwright (real browser) **Run:** `npm test` — see §8 for why never `npx vitest` directly.

---

1. What "working" means for this product

Before listing tests, it is worth stating what the suite is actually defending, because it decides what gets tested and what does not.

NestUp fails in three ways that matter, in descending order of cost:

1. **A member sees another member's data.** Unrecoverable. The whole trust proposition dies.
2. **A core flow silently does the wrong thing** — a message that does not arrive, a swiped room that comes back, a listing edit that also wipes the owner's chats.
3. **A screen is wrong or ugly.** Embarrassing, cheap to fix.

The suite is weighted to match. Access control and business rules are tested hard; presentation is tested where it encodes a rule (a button that must be disabled) rather than for its appearance.

What is deliberately not tested: every line of every component, exact copy, exact colours (except where a rule depends on them — see `map-basemap.test.ts`), and third-party libraries. The assignment says the main flows must be tested, not every line — and a suite that asserts on wording becomes a suite nobody dares change.

---

2. Feature tests — the main flows

Authentication and onboarding

TEST FILE	WHAT IT DEFENDS
`auth-form.test.tsx`	Sign-up and sign-in forms: field state, submit, error display
`signup-confirmation.test.ts`	Signing up must **not** hand back a usable session — the row is created, a code is mailed, and access waits for confirmation
`auth-password-actions.test.ts`	Forgot-password and reset flows
`auth-confirm-route.test.ts`	Every emailed link shape lands correctly: signup → onboarding, recovery → reset, invalid → `/login` with the right notice
`auth-email-templates.test.ts`, `auth-mail.test.ts`, `mail.test.ts`	The mail templates and sender behave
`password-input.test.tsx`	Show/hide toggle
`redirect.test.ts`	`sanitizeNextPath` — open-redirect hardening

Profile

TEST FILE	WHAT IT DEFENDS
`profile-action.test.ts`	Saving a profile writes the right rows and invalidates the right caches
`profile-required-fields.test.tsx`	Which fields block a save, and the banner that says so
`profile-form-sticky.test.tsx`	A rejected form keeps what the member typed (React 19 resets forms by default — this is the guard)
`about.test.tsx`, `contact-row.test.tsx`	Private details and contact visibility
`daily-life.test.tsx`, `daily-life-reminder.test.tsx`	The lifestyle questionnaire and its completion nudge
`profile-amenities.test.ts`, `profile-safe-room`, `interests-picker.test.tsx`	Preference inputs
`profile-avatar.test.tsx`, `photo-picker.test.tsx`, `photo-order.test.tsx`	Photo upload, ordering, lightbox
`profile-tabs-history.test.tsx`, `my-listing.test.tsx`	Profile tabs
`people.test.ts`	Another member's public profile

Listings

TEST FILE	WHAT IT DEFENDS
`listing-query.test.ts`	The browse query: filters, sort, pagination
`listing-actions.test.tsx`, `listing-status-action.test.ts`	Publish, pause, resume
`delete-listing-action.test.ts`	Soft removal
`mark-taken.test.tsx`, `listing-taken.test.ts`	"Taken" state
`listing-title.test.ts`, `property-tile.test.tsx`, `listing-gallery.test.tsx`	Presentation of a room
`filter-bar.test.tsx`, `sort-menu.test.tsx`	Filter and sort controls
`save-button.test.tsx`	Hearts, signed-in and signed-out
`roommate-tag-picker.test.tsx`, `roommate-count.test.ts`, `roommates-fit-rooms.test.ts`	Household composition rules
`co-posters.test.ts`, `co-posters-action.test.ts`, `co-poster-invites.test.tsx`	Co-ownership and invitations

Matching

TEST FILE	WHAT IT DEFENDS
`compatibility.test.ts`	The scoring function — each weighted component, and the "no preference ≈ 60%" convention
`swipe-rank.test.ts`	Deck ordering and the `MIN_DECK_SCORE` gate
`swipe-deck.test.tsx`	The deck UI, like/skip, empty state
`affinity.test.ts`	Interest overlap
`apartment-prefs.test.ts`, `no-city-prompt.test.tsx`	The preferred-city requirement and its prompt
`seed-data.test.ts`	**Measures the real deck across all 124 cities** so a town cannot silently become unmatchable

Chat and viewings

TEST FILE	WHAT IT DEFENDS
`send-message-action.test.ts`	Sending, and idempotent retry of the same `client_id`
`chat-realtime.test.tsx`	Token reaches the socket *before* the channel joins; one invalidation per burst
`chat-visible.test.ts`	Per-member "delete chat" cutoff
`chat-outbox.test.ts`, `message-composer.test.tsx`, `chat-media.test.ts`, `chat-format.test.ts`	Optimistic send, attachments, formatting
`message-owner.test.tsx`	"Message the household" entry point
`viewing-card.test.tsx`, `viewing-details.test.tsx`, `availability.test.ts`, `calendar.test.ts`	Proposing, approving, availability windows, calendar export

Settings and moderation

TEST FILE	WHAT IT DEFENDS
`account-actions.test.ts`, `account-section.test.tsx`	Email/password change
`danger-zone.test.tsx`	Account deletion, **including the listing-handover picker in all three shapes** (no heir, one heir, several)
`setting-toggle.test.tsx`, `settings-gear.test.tsx`	Settings controls
`moderation.test.ts`	Report, block, unblock, suspension
`notify.test.ts`	New-match notification

3. Invalid-input tests

AREA	WHAT IS ASSERTED
`validation.test.ts`	Every Zod schema: required fields, lengths, ranges, enums, coercion
`profile-required-fields.test.tsx`	A save missing a required field is refused with a field-level message
`send-message-action.test.ts`	Empty message, over-length message, malformed conversation id, a photo path outside the conversation's own folder
`listing-query.test.ts`	Nonsense query strings fall back to defaults rather than erroring (`.catch()` on every filter)
`amount-input.test.tsx`, `phone-field.test.tsx`, `date-picker.test.tsx`, `city-combobox.test.tsx`	Rejecting non-numeric rent, malformed phone numbers, impossible dates, unknown cities
`photo-rules.test.ts`, `photo-check.test.ts`	Wrong file type, oversized file, and a photo whose **content** does not match its declared tag
`redirect.test.ts`	`?next=https://evil.example` and other off-site targets are rejected
`image-client.test.ts`	Client-side compression and HEIC conversion boundaries

4. Business-process tests

These assert the rules that make the product itself correct, not just the code.

RULE	WHERE
A swiped room never returns to the deck	`swipe-rank.test.ts`, `cache-invalidation.test.ts`
Only rooms scoring ≥ 60 enter the deck	`swipe-rank.test.ts`
A seeker with no preferred city gets no deck	`apartment-prefs.test.ts`
One person, one active listing	`roommates-fit-rooms.test.ts`, `co-posters.test.ts`
Roommate count cannot exceed the rooms	`roommates-fit-rooms.test.ts`
Deleting an account **hands over** a shared listing rather than destroying it	`danger-zone.test.tsx`
A member with a live listing cannot inherit a second one	`danger-zone.test.tsx`
Deleting a chat hides it for one side only; the next message brings it back	`chat-visible.test.ts`
A blocked member disappears from decks and search, both ways	`moderation.test.ts`
A retried message with the same `client_id` does not double-post	`send-message-action.test.ts`
Every city can fill a deck	`seed-data.test.ts`

Cache invalidation as a business rule

`cache-invalidation.test.ts` deserves its own note. It asserts the **blast radius** of every mutation — that hearting a room touches exactly `saved:<me>` and `profile:<me>` and nothing else; that no listing mutation reaches into chat; that no chat mutation reaches into the deck or the room list.

This exists because of a real defect: mutations used to answer every write with a fistful of `revalidatePath` calls, so editing a room threw away the member's chats and profile too. The test encodes the rule that replaced it.

5. Permission tests

The highest-value category, since the worst failure mode is cross-member data exposure.

TEST	WHAT IT DEFENDS
`cache-invalidation.test.ts` → *"every per-member tag is scoped to the member who acted"*	Every `deck:` / `profile:` / `saved:` / `chat:` tag carries the acting member's id. A tag that forgot it would be **one shared cache key for the whole app** — the exact shape of a cross-user leak
`cached-session-boundary.test.ts`	**No Server Action may authorise itself with the cached session.** Identity is cached for *rendering* only; every write still does the uncached check. Both spellings compile, so only a test can hold this line
`cached-session-boundary.test.ts` (cont.)	The suspension check still reads the `suspensions` table and both gates still redirect on it
`signout-clears-cache.test.ts`	Logging out answers **303**, forcing a full document load — the only thing that empties the browser-held private caches. Also asserts there is **no GET handler**, so logout cannot be triggered cross-site
`member-actions.test.tsx`	The Log out button posts to the route handler, not a Server Action (a soft redirect would leave the caches alive)
`moderation.test.ts`	A member cannot lift their own suspension; blocking is mutual
`profile-action.test.ts`	Only the owner can edit a profile
`co-posters-action.test.ts`	Only an invited member can accept an invitation

The layer these cannot reach: RLS itself is enforced by Postgres, and the unit suite mocks the Supabase client. RLS is verified by the real-browser checks in §7 and by the policies being the *only* path to data — see the Security document.

6. Database tests

TEST	WHAT IT DEFENDS
`seed-data.test.ts`	A **sha256 fingerprint** over the first 92 seed records. Re-running `npm run seed` can never silently change existing demo data
`seed-data.test.ts` (cont.)	Re-measures deck coverage across all 124 cities with the real `buildDeck`
`city-centres.test.ts`	Guards all 124 generated city coordinates against drift
`listing-query.test.ts`	The filter → SQL translation, including pagination `range()` maths
`cache-lifetimes.test.ts`	Every `cacheLife` declares `stale ≥ 300s` (the App Shell threshold), and the router cache is not shorter than the data caches
`roommates-fit-rooms.test.ts`, `roommate-count.test.ts`	The trigger-maintained denormalised household columns

7. Edge cases

CASE	TEST
Empty deck — every room already swiped, or none scores 60	`swipe-deck.test.tsx`, `swipe-rank.test.ts`
Empty inbox, and a chat deleted then revived by a new message	`chat-visible.test.ts`
A room removed while a conversation about it is open	`listing-taken.test.ts`, and the second `listings` SELECT policy
Concurrent conversation creation (unique-constraint race)	`findOrCreateConversation` falls back to re-reading
A duplicate message send	`send-message-action.test.ts`
Account deletion with 0 / 1 / several eligible heirs	`danger-zone.test.tsx`
Realtime socket joining before the token arrives	`chat-realtime.test.tsx`
A failing realtime sync must not throw inside an effect	`chat-realtime.test.tsx`
Stale `?needs=cities` in the URL after the member fixed it	`no-city-prompt.test.tsx`
A listing with no photos	`listing-gallery.test.tsx`
Navigation must stay in one tab	`same-tab-navigation.test.ts` + `npm run check:nav`

Real-browser checks (Playwright)

Some things cannot be asserted in jsdom — they need a real engine, a real GPU canvas, or the real deployment.

SCRIPT	WHAT IT PROVES
`npm run check:map`	Both themes render; counts red pixels **straight off the WebGL canvas** (a GL layer has no DOM to query)
`npm run check:nav`	No internal link opens a new tab
`.superpowers/e2e/nav-cache.mjs`	**21 assertions on the live site:** logout destroys the JS context; `/swipe` redirects after logout; the back button cannot restore a signed-in page; a second member in the same tab sees only their own data
`.superpowers/e2e/chat-freshness.mjs`	A real message reaches the thread *and* updates the cached inbox row, and survives navigating away and back
`.superpowers/e2e/skeleton-duration.mjs`	Measures how long a loading skeleton is actually on screen, per tab

8. How to run

```
npm test                              # 97 files, 740 tests
npm test -- --testTimeout=25000       # use this if userEvent tests time out (see below)
npx tsc --noEmit                      # types
npx eslint                            # lint
```

Two environment traps, both learned the hard way and both worth knowing before believing a red result:

1. **Always `npm test`, never `npx vitest run`.** The npm script sets `NODE_OPTIONS=--no-experimental-webstorage`. Without it, Node 25 installs its own inert `localStorage` global that jsdom does not replace, and `theme-toggle.test.tsx` fails with `localStorage.clear is not a function` — a failure that says nothing about the component.
2. **OneDrive sync starves `userEvent`.** The project lives in a synced folder; when OneDrive is busy, half a dozen interaction tests exceed the 5 s default. Re-run with `--testTimeout=25000` before treating a timeout as a real failure.

9. Known gaps

Stated plainly, because a test document that claims full coverage is not credible.

- **RLS policies are not exercised by the unit suite.** It mocks the Supabase client, so the policies themselves are covered only by real-browser checks and by review. A future improvement is a test that runs real queries as two different members against a test project.

- **No automated end-to-end journey.** Playwright is used for targeted checks, not a full signup → profile → swipe → chat → viewing script.
- **Email delivery is mocked.** Templates and the sender are tested; actual inbox arrival is verified by hand.
- **The Gemini photo check is mocked.** Its contract is tested; the model's judgement is not.
- **Load has not been measured.** Scale reasoning is analytical — see Scale.
